

Reviewer Pack — Minimal Rank of Formal Power Series vs DPLL Proof Width for ...

Entry 9e6d4f5f4a9e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 23:31:59 UTC

Minimal Rank of Formal Power Series vs DPLL Proof Width for Tseitin Formulas

Entry ID: 9e6d4f5f4a9e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 23:31:59 UTC

1. Conjecture

Field A (mathematical branch): Formal Power Series Theory **Field B** (complexity object): Complexity Theory: DPLL Proof Complexity (for Tseitin Formulas)

Statement:

[‘For every Tseitin formula F with n variables and m clauses, let $R(F)$ be the minimal rank of the monomial formal power series representing the incidence matrix of F . Then, for all instances F , the DPLL proof width $w(F)$ satisfies $w(F) = \Theta(R(F))$.’, ‘Equivalently, there exists a constant $c > 0$ such that for all Tseitin formulas F with n variables and m clauses, $R(F) \leq cw^*(F)$.’]

Rationale (proposer’s reasoning):

[‘Formal power series provide a rich algebraic structure to encode combinatorial objects. The minimal rank of a formal power series may capture the complexity of encoding information in a Tseitin formula, which could be related to the DPLL proof width.’, ‘Since Tseitin formulas are used in resolution proofs for SAT, any structural property that can be captured by formal power series might have implications for the complexity of such proofs.’]

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 496db6564c71e504

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For the conjecture that DPLL proof width is $\Theta(R(F))$, support is indicated if, for all 30 seeds, the mean ratio of $R(F)$ to $w^*(F) \leq 1.2$ and the standard deviation of this ratio is ≤ 0.1 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_tseitin_formula(n):
        variables = [f'x{i}' for i in range(1, n+1)]
        clauses = []
        for var in variables:
            clauses.append([var])
        for i in range(2, n+1):
            clauses.append([f'x{i}', f'~x{i-1}'])
        return variables, clauses

    def incidence_matrix(variables, clauses):
        m = len(clauses)
        n = len(variables)
        matrix = [[0] * (n + 1) for _ in range(m)]
        for i, clause in enumerate(clauses):
            for var in clause:
                if var.startswith('~'):
                    j = int(var[2:]) - 1
                    matrix[i][j+1] = -1
                else:
                    j = int(var[1:]) - 1
                    matrix[i][j+1] = 1
        return matrix

    def min_rank(matrix):
        m, n = len(matrix), len(matrix[0])
        rank = 0
```

```

for i in range(n):
    if all(matrix[j][i] == 0 for j in range(m)):
        continue
    rank += 1
    for j in range(m):
        if matrix[j][i] != 0:
            pivot_row = j
            break
    for k in range(n):
        matrix[pivot_row][k], matrix[i][k] = matrix[i][k], matrix[pivot_row][k]
    for j in range(m):
        if j == i:
            continue
        factor = -matrix[j][i] / matrix[i][i]
        for k in range(n):
            matrix[j][k] += factor * matrix[i][k]
return rank

def dpll_width(clauses, assignment):
    stack = []
    while True:
        if all(c in assignment for c in clauses):
            return len(assignment)
        unit_clause = next((c for c in clauses if len([x for x in c if x not in assignment and not (~x in assignment)]))
        if unit_clause is None:
            return float('inf')
        var = [v for v in unit_clause if v not in assignment and '~' + v not in assignment][0]
        assignment[var] = True
        stack.append((var, False))
        while stack:
            var, negated = stack.pop()
            if negated:
                del assignment[var]
            else:
                assignment['~' + var] = True
                for c in clauses:
                    if all(x not in assignment and '~' + x not in assignment for x in c):
                        continue
                    if any(x in assignment and assignment[x] == (not negated) for x in c):
                        stack.append((var, True))
                        break
                else:
                    stack.append((var, False))

n = random.choice([5, 10, 15, 20, 30, 40])
variables, clauses = generate_tseitin_formula(n)
matrix = incidence_matrix(variables, clauses)
R_F = min_rank(matrix)

assignment = {}
w_F = dpll_width(clauses, assignment)

if w_F == float('inf'):
    return {
        "metric_name": "R(F) / w*(F)",
        "metric_value": None,

```

```

        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "DPLL proof width is infinite"
    }

ratio = R_F / w_F
return {
    "metric_name": "R(F) / w*(F)",
    "metric_value": ratio,
    "instances_tested": 1,
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        if result["metric_value"] is not None:
            results.append(result["metric_value"])

    mean_ratio = sum(results) / len(results)
    std_ratio = math.sqrt(sum((x - mean_ratio)**2 for x in results) / len(results))
    support_fraction = sum(1 for r in results if r <= 1.2) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_ratio} support_fraction={support_fraction}")
    elif any(r > 1.2 for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if result > 1.2)
        print(f"RESULT: FALSIFIED counterexample='R(F) / w*(F) > 1.2' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient support for conjecture")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_598cf2aa.py", line 123, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_598cf2aa.py", line 97, in run_trial
    w_F = dpll_width(clauses, assignment)
           ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_598cf2aa.py", line 68, in dpll_width
    if all(c in assignment for c in clauses):
        ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_598cf2aa.py", line 68, in <genexpr>

```

```
if all(c in assignment for c in clauses):
    ^^^^^^^^^^^^^^^^^
```

TypeError: unhashable type: 'list'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Investigate and fix the crash in the test code to continue testing the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11968
2	propose	ollama_remote	glm4:latest	0	0	10904
3	preregistration	ollama_remote	glm4:latest	0	0	5684
4	novelty	ollama_remote	glm4:latest	0	0	4649
5	novelty	ollama_remote	glm4:latest	0	0	5204
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15494
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12303
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14068
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13642
10	judge	ollama_remote	glm4:latest	0	0	10389

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 104305 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/9e6d4f5f4a9e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9e6d4f5f4a9e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/9e6d4f5f4a9e.tar.gz` (if generated)